REFLEKSI SINGKAT MLOPS:
URGENSI PRINSIP 'CODE AS A TRIGGER' DALAM MENCEGAH KEGAGALAN SISTEM DI PRODUKSI (SILENT FAILURES)

Studi Kasus: Sistem Prediksi Skor Viralitas dan Deteksi Kejenuhan Konten Media Sosial (ViralSense)

Penyusun: Galih Praditya Kurniawan	**Dosen Pengampu:** Rizal Setya Perdana, S.Kom., M.Kom., Ph.D.
NIM: 245150201111017	**Mata Kuliah:** Machine Learning Operations (MLOps)
Program Studi: Magister Ilmu Komputer	**Institusi:** Fakultas Ilmu Komputer, Universitas Brawijaya

Intisari Refleksi: *Berbeda dari perangkat lunak tradisional di mana cacat logika menimbulkan pesan kesalahan fatal (crash/HTTP 500), kegagalan pada sistem Machine Learning sering kali bersifat senyap (**silent failures**)—layanan tetap mengembalikan status HTTP 200 OK namun menghasilkan prediksi yang rusak, terdegradasi, atau bahkan acak. Alur kerja otomatisasi **'Code as a Trigger'** menjadi benteng pertahanan krusial untuk mencegah silent failures melalui verifikasi kode otomatis (pytest), pelatihan ulang deterministik dengan silsilah data terkelola (DVC), evaluasi ketat berbasis ambang batas teknis (LK-01), dan promosi berjenjang pada Model Registry (Staging).*

1. Fenomena Silent Failures dalam Machine Learning Produksi

Dalam paradigma rekayasa perangkat lunak konvensional (Software Engineering 1.0), integritas sistem didominasi oleh kebenaran sintaksis dan alur eksekusi logika kode program. Jika terjadi regresi seperti dereferensi pointer null, kesalahan tipe variabel, atau pengecualian basis data, sistem akan secara instan melempar *loud error* (seperti kode status HTTP 500, crash process, atau log exception traceback) yang langsung disadari oleh operator melalui sistem monitoring kesehatan server.

Sebaliknya, pada sistem Machine Learning (Software 2.0), model inferensi pada dasarnya hanyalah serangkaian perkalian matriks bobot matematis. Ketika kode pra-pemrosesan diubah secara ceroboh, normalisasi fitur bergeser, atau hyperparameter yang tidak tepat di-commit ke repositori, model yang dilatih ulang tetap dapat dimuat oleh serving container (FastAPI/Triton) dan tetap merespons permintaan klien dengan respons HTTP 200 OK. Namun secara substansial, output prediksi skor viralitas yang dihasilkan menjadi tidak akurat, bias, atau bahkan terdegradasi menjadi tebakan acak. Inilah yang diistilahkan sebagai **Silent Failures**.

Pada kasus proyek **ViralSense**—platform analitik waktu-nyata yang memprediksi momentum viralitas video pendek YouTube Shorts—konsekuensi dari silent failure sangat fatal bagi agensi pemasaran digital dan kreator. Kreator konten yang menginvestasikan jutaan rupiah untuk memproduksi video berdasarkan rekomendasi model yang cacat akan mengalami kegagalan distribusi audiens, rasio kegagalan konten membengkak di atas 25%, dan keputusan operasional menjadi tersesat tanpa disadari oleh tim teknis selama berminggu-minggu.

2. Anatomi Akar Penyebab Silent Failures pada Pipeline ML

Berdasarkan analisis arsitektur end-to-end ViralSense, terdapat empat simpul kerentanan utama yang dapat memicu terjadinya kegagalan senyap apabila alur kerja tidak diotomatisasi secara ketat:

a. Perubahan Logika Rekayasa Fitur & Kebocoran Data (Feature Engineering Drift): Penyesuaian skrip imputasi nilai kosong (*clean_missing_values*) atau formula ekstraksi rasio (misalnya rasio komentar terhadap like atau laju kecepatan penayangan) dapat secara tidak sengaja memodifikasi skala input inference. Jika kode transformasi training tidak selaras dengan inference serving, model akan memproses fitur dengan makna numerik yang cacat (*training-serving skew*).

b. Degradasi Tersembunyi Akibat Hyperparameter Suboptimal: Modifikasi kecil pada parameter regularisasi (seperti penurunan kedalaman pohon max_depth yang terlalu agresif atau perubahan learning_rate) dapat menyebabkan model mengalami underfitting atau overfitting parah. Tanpa evaluasi ambang batas otomatis, model

yang performanya anjlok ini dapat secara tidak sengaja terdorong ke produksi menggantikan model Champion yang sudah terbukti stabil.

c. Pergeseran Data dan Latensi Label (Data & Concept Drift): Dinamika tren video pendek berubah dalam siklus mingguan. Apabila model dilatih menggunakan snapshot data usang yang tidak terhubung dengan silsilah versi DVC yang sahih, model baru akan kehilangan kepekaan terhadap pola audio latar dan slang terbaru di YouTube Shorts.

3. Mengapa Prinsip 'Code as a Trigger' Merupakan Solusi Kritis

Prinsip **'Code as a Trigger'** mendikte bahwa setiap modifikasi pada basis kode—sekecil apa pun perubahannya—harus diposisikan sebagai hipotesis yang belum terbukti dan wajib diverifikasi oleh orkestrator CI/CD secara otomatis tanpa toleransi intervensi manual. Pada implementasi ViralSense melalui GitHub Actions (.github/workflows/mlops-automation.yaml), sistem membangun empat lapis pertahanan (four-tier defenses):

Tahap 1: Verifikasi Integritas Kode Otomatis (Pytest Fail-Fast Gate)

Sebelum sumber daya komputasi dialokasikan untuk melatih model, job automated-testing mengeksekusi suite uji komprehensif pada modul pra-pemrosesan data, validasi skema, factory model, dan fungsi metrik. Jika ada perubahan sintaksis atau regresi fungsional yang merusak kontrak masukan/keluaran, pipeline langsung dihentikan seketika (*fail-fast*), mencegah terbuangnya waktu dan kuota retraining.

Tahap 2: Pelatihan Ulang Otomatis dengan Reproduksibilitas Data DVC

Melalui orkestrasi train.py, retraining dijalankan menggunakan dataset fitur yang terlacak silsilahnya secara deterministik via DVC (Data Version Control). Setiap baris data latih dipetakan ke hash MD5 data raw yang tersinkronisasi ke Cloudflare R2 Object Storage. Hal ini mengeliminasi anomali 'model bekerja di komputer lokal tetapi gagal di produksi' karena variasi versi dataset lokal.

Tahap 3: Gerbang Evaluasi Kuantitatif Berbasis Ambang Batas LK-01

Tahap inilah yang menjadi penangkal utama silent failures. Modul src/models/evaluate.py secara ketat menguji kandidat model baru terhadap ambang batas teknis yang telah dibakukan dalam dokumen **LK-01 Tabel 4.1**:

• **Target Akurasi $\ge 82\%$ (0.82)**: Memastikan ketepatan klasifikasi umum terhadap populasi video.
• **Macro F1-Score ≥ 0.80**: Menjamin model seimbang dan tidak bias ke kelas mayoritas non-viral.
• **Latensi Inferensi Single-Sample $< 200 ext{ ms (p95)}$**: Memenuhi Service Level Agreement (SLA) layanan inferensi real-time.
• **ROC-AUC ≥ 0.85**: Memastikan pemisahan skor probabilitas antar kelas viral vs non-viral sangat tajam.

Jika satu saja ambang batas ini dilanggar (misalnya F1-Score turun ke 0.77 akibat perubahan hyperparameter yang buruk), skrip evaluasi melempar sinyal kegagalan (exit code 1), menandai run sebagai FAILED, dan membatalkan seluruh proses selanjutnya. Tidak ada model cacat yang dapat lolos secara senyap.

Tahap 4: Pembaruan Model Registry Berjenjang (Stage: Staging)

Model yang berhasil membuktikan keunggulannya tidak langsung dipublikasikan ke trafik pengguna (Production), melainkan secara otomatis didaftarkan ke MLflow Model Registry dengan stage **Staging**. Di tahap ini, metadata silsilah (commit SHA, branch, DVC hash, dan metrik hasil uji) diikat secara permanen pada model version. Hal ini memberikan transparansi audit yang menyeluruh dan memungkinkan tim teknis melakukan Root Cause Analysis (RCA) dalam hitungan menit jika terjadi anomali di masa depan.

4. Matriks Komparasi: Moda Kegagalan vs Mekanisme Proteksi Otomatis

Moda Silent Failure	Manifestasi Tanpa Otomatisasi	Mekanisme Pencegahan Otomatis	Hasil Akhir Proteksi
Regresi Kode Pra-pemrosesan	Fungsi imputasi mengembalikan nilai NaN atau tipe data salah. Inferensi tetap jalan dengan imputasi default 0.	Job automated-testing (pytest) mengecek skema, imputasi numerik, dan tokenisasi.	■ Build dibatalkan pada Tahap 1 sebelum retraining dimulai.
Degradasi Performa Model (Underfit/Overfit)	Hyperparameter buruk menyebabkan F1-Score turun drastis. Model cacat dideploy manual karena 'tidak ada error'.	Gerbang evaluasi otomatis (evaluate.py) menguji metrik terhadap ambang batas ketat LK-01 (F1 ≥ 0.80, Akurasi $\ge 82\%$).	■ Model ditolak (FAILED), tidak didaftarkan ke Staging.
Pelanggaran Batas Latensi (SLA Spike)	Arsitektur pohon terlalu dalam membuat waktu inferensi membengkak ke 350 ms, melumpuhkan gateway API di bawah beban tinggi.	Uji latensi empiris 20 iterasi pada skenario riil YouTube Shorts memverifikasi latensi p95 $< 200 ext{ ms}$.	■ Menjamin model yang dipromosikan ke Staging selalu siap melayani beban real-time.
Ketidakpastian Asal-usul Model (Lineage Loss)	Model dideploy dari laptop engineer tanpa catatan kode commit, dataset, atau hyperparameter mana yang dipakai.	MLflow Model Registry otomatis mengikat model version dengan Git commit SHA dan DVC dataset MD5 hash.	■ 100% auditabilitas dan reproduksibilitas eksperimen terjamin.

5. Kesimpulan dan Pembelajaran

Demonstrasi prinsip **'Code as a Trigger'** pada proyek ViralSense membuktikan bahwa otomatisasi bukan sekadar kemudahan operasional (convenience), melainkan prasyarat mutlak untuk menjaga keandalan sistem kecerdasan buatan pada level produksi. Dengan mengalihkan beban verifikasi dari intuisi manual manusia ke alur kerja berbasis kode yang deterministik, tim pengembang dapat melakukan iterasi eksperimen secara cepat dan percaya diri tanpa kekhawatiran merusak kualitas layanan yang sedang berjalan.

Pemisahan tanggung jawab antara tahap pengujian kode (unit testing), retraining bersilsilah (DVC), gerbang validasi ambang batas (LK-01 Evaluation Gate), dan promosi bertahap ke MLflow Staging menutup celah terjadinya kegagalan senyap (silent failures) di seluruh siklus hidup model, mewujudkan sistem MLOps yang tangguh, efisien, dan berintegritas tinggi.